

Box Transpose: In-Place N-Dimensional Tensor Permutation on GPUs

Kaveen Jayamanna Alexis Newman

Kennesaw State University

CS 8045 Advanced Algorithms — Project Proposal

Team and Responsibilities

Kaveen Jayamanna (KJ). The planner and its cost model (`src/_plan.py`): the box-swap graph, the shortest-path search, and the derivation of τ , μ and D_{post}^* . The in-place cycle-following CUDA kernel (`src/_kernels/`), including the scalar and vectorised paths.

Alexis Newman (AN). Measurement and validation: the throughput micro-benchmark that calibrates the cost model on the evaluation hardware, the correctness suite that gates every result, the end-to-end comparison against the out-of-place baseline, and the head-to-head against the closest prior systems.

Both members share the write-up and the final presentation. Table 2 assigns every milestone to one of them.

[PLACEHOLDER] Confirm this division before submitting. It is a proposed split based on where each part of the work already sits, not one that has been agreed.

1 Statement of the Problem

1.1 Background

Modern deep learning workloads run close to the capacity limit of GPU memory. The pressure comes from model parameters and activations, and also from intermediate tensor transformations. Tensor layout conversion is a common example.

Operations such as `permute` [7] and `transpose` [8] in PyTorch [9] reorder the logical axes of a tensor without moving any values. They only change the tensor’s metadata, so they are free. What they leave behind is a strided layout that is no longer contiguous.

Many downstream operations require a contiguous layout, and others merely run slower on a strided one. A layout mismatch between what one operation produces and what the next one needs is therefore routine. A recent PyTorch RFC

reports that non-contiguous tensors arise across sequence, tensor and pipeline parallelism, and that several communication primitives require contiguous inputs [12].

1.2 The cost of the standard fix

PyTorch resolves a mismatch with `.contiguous()` [6]. It allocates a new tensor and copies every element into its new position. The copy is attractive because it is highly parallel. Each output position can be computed independently, so the GPU can use all of its threads.

The cost is memory. The original tensor and the new one must both exist until the copy finishes. A single conversion can therefore approach twice the footprint of the tensor being converted. On a large tensor that is enough to cause an out-of-memory failure, and the short-lived allocation also fragments the GPU heap [10, 12].

The workaround turns a layout problem into a peak-memory problem. Our goal is to remove the second buffer instead, by converting the tensor in place.

1.3 The problem, formally

Let A be a tensor of rank r with shape $(d_0, \dots, d_{r-1})$, stored contiguously in row-major order, and let π be a permutation of the r axis positions. The *in-place permutation problem* is to rewrite the buffer holding A so that it holds the tensor A^π with shape $(d_{\pi(0)}, \dots, d_{\pi(r-1)})$, contiguous in the same row-major order, using auxiliary space $o(N)$ where $N = \prod_i d_i$.

The constraint that makes this hard is that no value may be overwritten before it has been read. The classical solution is cycle following [3, 5]. A permutation decomposes into disjoint cycles, and rotating the values along each cycle moves every element exactly once with no second buffer. Cycle following is a poor fit for a GPU, because cycle discovery is sequential and the resulting accesses are irregular [1, 4].

1.4 Objectives

1. Decompose an arbitrary π into a sequence of operations, each of which is a single in-place matrix transpose. The cycle-following machinery that is well understood in two dimensions then applies unchanged at every step.
2. Choose the decomposition that moves the fewest bytes, rather than accepting the first one found, by searching the space of decompositions under an explicit cost model.
3. Keep auxiliary space independent of N . The only per-step state is the cycle metadata, which must stay at kilo-byte scale for tensors of any size.
4. Beat the memory ceiling of `.contiguous()` on real shapes, and be no slower than it where both fit.

2 Problem Difficulty Analysis and Proposed Method

2.1 Why this is hard

Three things make in-place permutation of an N-D tensor harder than in-place matrix transposition.

2.1.1 The permutation is not a single transpose

Cycle following works on a matrix because the index map has a closed form. Position p of an $m \times n$ matrix moves to $(p \bmod n)m + \lfloor p/n \rfloor$, and the orbits of that map partition the buffer. An arbitrary π over r axes induces an index map with no such closed form. It can still be decomposed into disjoint cycles, because every permutation can [3]. But computing those cycles means materialising the map. That is $\Theta(N)$ work and $\Theta(N)$ state, which defeats the purpose.

2.1.2 The decomposition is not unique, and the choices are not equal

Any π can be reached by a sequence of simpler moves, and there are many such sequences. They are not interchangeable. Each move rewrites the whole buffer, so a decomposition of k steps pays roughly k passes over memory. A decomposition that is short in steps may still move more bytes than a longer one, because what a step actually moves depends on how many positions it leaves fixed. Choosing well is a search problem rather than a bookkeeping one.

2.1.3 Memory access, not arithmetic, is the cost

A permutation does no arithmetic. Every step is bound by how the memory system responds to the access pattern that step produces. That response is strongly non-linear in the stride. A step whose innermost contiguous run is long enough to fill a cache line runs at a large multiple of the throughput

of one whose run is short. A cost model for this cannot be derived from operation counts. It has to be calibrated against the hardware.

The search space is $r!$ orderings of the axes, with an edge between two orderings for every move that connects them. An exhaustive comparison is out of reach at the ranks that matter.

2.2 Proposed method: adjacent box swaps

We propose decomposing π into a sequence of *adjacent box swaps*.

A *box* is one or more neighbouring axes treated as a single unit. Those axes are adjacent in a row-major layout, so their extents multiply into one logical extent and the group behaves exactly like a single axis of that size. A *swap* exchanges two adjacent boxes.

The point of the construction is what a swap reduces to. Fix a split of the axis list into a prefix, a left box, a right box and a suffix, with extents D_{pre} , $rows$, $cols$ and D_{post} . Exchanging the two boxes is exactly an in-place transpose of a logical $rows \times cols$ matrix. It is performed independently for each of the D_{pre} leading positions, and each element of it is a contiguous run of D_{post} values. Every step of the decomposition is therefore a problem we already know how to solve in place, and the two-dimensional cycle-following result applies unchanged at every step.

D_{post} also determines whether a step is fast. A large D_{post} means each moved element is a long contiguous run, which is what the memory system rewards. When it is divisible by the vector width, the kernel moves four values at a time instead of one.

2.3 Choosing the decomposition by shortest path

A permutation can always be reached by bubbling each axis into position with single-axis swaps. Treating a run of adjacent axes as one box instead turns the decomposition into a shortest-path problem over axis orderings. The search is then free to spend one swap where the baseline spends several.

We build a graph whose vertices are orderings of the axes and whose edges are legal box swaps. Each edge is weighted by the bytes that swap moves, divided by a calibrated throughput. Dijkstra from the identity ordering to π then returns the cheapest decomposition under the cost model rather than merely a correct one.

To measure what that freedom is worth we sample shape and permutation pairs from the space defined in Section 3.2 and plan each of them both ways. The sweep covers five ranks, 2 through 6, at 40,320 elements throughout. It draws 12 shapes per rank and up to 4 permutations per shape, giving 204 pairs and 408 plans. The comparison is paired. Both planners see the same shape and the same permutation, and

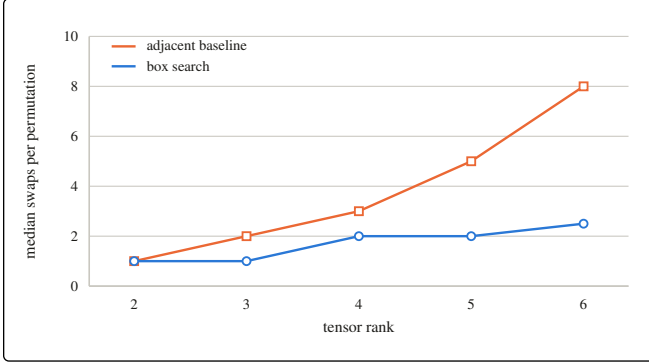


Figure 1: **Median swaps per permutation against tensor rank.** 5 ranks, both planners on the same shapes. The baseline’s cost climbs with the number of adjacent transpositions a permutation needs. The search stays flat. At rank 6 it ends at 2.5 swaps against 8.

every rank holds the element count fixed, so a difference in bytes moved is a difference in the decomposition rather than in the tensor.

Across the sweep the search issues 2.0x fewer swaps than the adjacent baseline and moves 2.0x fewer bytes at the median. It reaches 5.6x on the permutation where it helps most. The margin widens with rank, as Figure 1 shows. At rank 6 the median permutation costs the baseline 8 swaps and the search 2.5. Table 1 gives the breakdown. 65% of the swaps the search selects bundle more than one axis into a box. That is the move the baseline cannot make, and it is where the difference comes from.

The auxiliary cost of a plan is its cycle metadata, and it does not scale with the tensor. The largest plan in the sweep needs 3,728 bytes, which is 2.31% of the data it permutes. The data itself is never copied to a second buffer. That is the objective the whole construction exists to serve.

2.4 Feasibility

A working prototype exists, and the numbers above come from it rather than from an estimate. The planner is implemented in `src/_plan.py` and the cycle-following kernel in `src/_kernels/box_kernel.cu`, with both a scalar and a vectorised path. What remains is calibration and evaluation rather than construction. That is why the timeline in Section 4 is weighted towards measurement.

Edge weight is bytes moved divided by a calibrated throughput μ , currently 69.7 GB/s. Two thresholds gate the search. τ is the smallest D_{post} at which an edge is worth considering. D_{post}^* is the point at which box following beats a cycle splitter. Both currently sit at 1 and 1, so the pruning they describe is disabled. Re-deriving them on the evaluation hardware is a milestone in its own right.

[PLACEHOLDER] One known gap to close before the final report. During the search `estimated_cycle_info` reports no fixed points, so every edge weighs the same and the search minimises swap count rather than bytes. Exact counts are filled in afterwards, so the reported numbers are accurate for the plans chosen, but they are not what the search optimised. Quantify the resulting gap.

2.5 Why this improves on existing work

We compare against prior work along the dimensions that matter for this problem. These are the number of full-tensor passes, the auxiliary memory a method needs, how sensitive its cost is to the tensor shape, and whether it supports arbitrary N-D permutations.

Data movement. Most GPU-oriented approaches trade extra data movement for regular, well-behaved kernels [1, 2, 4, 13]. They may read and write the tensor several times even when the permutation itself requires moving each element once. Cycle following is the exception, since it moves each element exactly once [5], but the original formulation is built around individual cycles and does not give an efficient GPU implementation for arbitrary N-D permutations. Our proposal sits with the first group and accepts multiple passes. Its claim is that it takes fewer of them than a fixed decomposition rule does, and the sweep above is the evidence.

Auxiliary memory. Existing methods size their temporary storage from the tensor shape or from the decomposition strategy. Catanzaro et al. [1] need storage related to the larger dimension. Gomez-Luna et al. [4] may pad an axis to obtain a usable factorization, so the workspace depends on the input. ITTPD [2] exposes a memory-budget parameter, but uses it to set how many batch instances its primitive kernels process. Our auxiliary state is the cycle metadata of the selected steps, which the sweep measures at 2.31% of the tensor in the worst case.

Shape sensitivity. Several of these designs rely on cache locality to reach peak throughput [1, 2, 4]. A shape that yields short or irregular accesses costs them disproportionately. A search over decompositions attacks this directly, because the cost model weights an edge by the access pattern that edge produces. Whether that translates into more stable measured runtime is an open question and is a deliverable of the evaluation.

Permutation coverage. Catanzaro et al. [1], Gustavson et al. [5] and Gomez-Luna et al. [4] define their kernels for two-dimensional permutations or single axis swaps. Data layout transformation systems apply the same machinery to strided layouts [11]. None of them addresses an arbitrary permutation of an arbitrary rank directly. EITHOT [13] and ITTPD [2] do, and ITTPD decomposes the permutation before transposing it, which is the same overall shape of idea as ours.

What is different here. Against ITTPD in particular, our proposal differs in two ways, and these are the claims we intend to defend.

Table 1: **What the box search saves over the adjacent baseline, by rank.** 204 shape and permutation pairs across 5 ranks, sampled from the shape space of Equation (1) and planned by both strategies at a fixed tensor size. The bytes-moved advantage grows with rank, from 1.0x where a permutation is a single transpose to 2.9x at rank 6.

Rank	$\Omega(N, r)$	Shapes	Pairs	Swaps		Bytes	Aux (B)
				Adj.	Search		
2	96	12	12	1	1	1.0x	304
3	1,944	12	48	2	1	1.5x	592
4	19,200	12	48	3	2	2.0x	3,728
5	123,750	12	48	5	2	2.1x	2,296
6	598,752	12	48	8	2.5	2.9x	720

$\Omega(N, r)$ is the size of the shape space of Equation (1) at this rank. **Shapes** are drawn from it uniformly and **Pairs** counts (shape, permutation) cases. Swaps and bytes are medians over the pairs in that row. **Bytes** is the median per-pair ratio of bytes moved by the adjacent baseline to bytes moved by the search, so higher is better. **Aux (B)** is the largest cycle-metadata footprint any one search plan in the row needed, in bytes.

1. **Boxes, not axes.** A step may exchange two *groups* of adjacent axes rather than two axes. This strictly enlarges the set of reachable decompositions. The measurements above show the search takes that option on 65% of the steps it selects, so it is not a generalisation that goes unused.
2. **Search under a calibrated cost model, not a fixed rule.** The decomposition is chosen by shortest path over a graph whose weights are measured throughput. The planner therefore adapts to the hardware it is calibrated on, instead of encoding one machine’s behaviour as a heuristic.

[PLACEHOLDER] Relationship to Cycle Splitter, our group’s other approach to the same problem, needs stating here with a citation. Cycle Splitter moves each element exactly once and bounds auxiliary memory explicitly. Box following accepts multiple passes in exchange for regular, coalesced kernels, and wins where D_{post} is large enough. The constant D_{post}^* in `src/_plan.py` is the routing gate between the two, so the comparison is not rhetorical. Deriving it is a milestone.

[PLACEHOLDER] A direct head-to-head against EITHOT and ITTPD on their reported shapes is the strongest evidence for these claims, and is not yet run. It is the deliverable of the evaluation milestone.

3 Simulation Setting

This is a programming-based project, so what follows is the implementation and measurement environment rather than a simulator.

3.1 Programming environment

The planner and cost model are Python, standard library only, so they run anywhere and are testable without a GPU. The kernel is CUDA C++, compiled on first use by PyTorch’s `cpp_extension` JIT and cached thereafter, and driven from Python through a thin `profile()` / `apply()` interface that uploads the cycle metadata once and then applies a plan repeatedly.

The whole environment is pinned in a container built from the NGC PyTorch image, which supplies torch, CUDA, `nvcc` and `ninja`. The repository ships the `Dockerfile` and a dev container definition, so the measurement environment is reproducible rather than described. Development hardware is an RTX PRO 6000 Blackwell.

3.2 Benchmark design

A fair evaluation of tensor layout conversion has to cover a broad range of tensor shapes rather than a small set of hand-picked examples. For a tensor of N elements and rank r , we define the shape space as every ordered r -tuple whose dimensions multiply to N . Candidate shapes are enumerated by prime-factorising N and distributing the prime factors across the dimensions:

$$s = \left(\prod_{i=1}^m p_i^{b_{i1}}, \prod_{i=1}^m p_i^{b_{i2}}, \dots, \prod_{i=1}^m p_i^{b_{ir}} \right). \quad (1)$$

Here $N = p_1^{e_1} p_2^{e_2} \dots p_m^{e_m}$, and b_{ij} is the amount of prime p_i assigned to dimension s_j , so that $\sum_j b_{ij} = e_i$ for every i . The number of ways to split one prime’s exponent across r dimensions is given by stars and bars, so the size of the space is

$$\Omega(N, r) = \prod_{i=1}^m \binom{e_i + r - 1}{r - 1}. \quad (2)$$

This is the same construction the Cycle Splitter evaluation uses, which keeps the two sets of results comparable. It also makes the comparison checkable. The same equation builds the full list for any N and any rank, so anyone can generate the cases we did not pick and run them against any method here, including ours.

We fix N at 40,320 elements and sweep ranks 2 through 6. From each rank’s space we draw 12 shapes uniformly, and for each shape we draw up to 4 non-identity axis permutations. The identity is excluded because the tensor is already contiguous under it. Shapes containing a unit axis are rejected, since an axis of extent one can be moved without touching a byte.

The space is large enough that sampling is the only option. At rank 6 it holds 598,752 shapes.

3.3 Data sources

Planner sweep. Every permutation at each rank from 2 to 6, planned by both strategies. CPU only, seeded, and committed to the repository, so every planner number in this proposal reproduces from a clean checkout with no GPU present. This is the source of Table 1 and Figure 1.

Throughput micro-benchmark. A sweep of D_{post} on the target GPU, used to locate the throughput knee τ and to fit μ . It is regenerated per machine and not committed, because it describes the hardware that produced it.

Pilot and benchmark sweeps. Wider runs over rank-2 through rank-6 shapes used to score the cost model against reality. These are hundreds of megabytes and are kept outside the repository, referenced through environment variables, with the scripts skipping rather than failing when they are absent.

Comparison shapes. For the head-to-head evaluation, the shapes reported by the closest prior systems [2, 13], so the comparison is on their ground rather than ours.

3.4 Metrics

Bytes moved. The primary cost metric, and what the cost model predicts. Counted exactly from the cycle structure of each step, so it is independent of the machine and can be compared against a plan produced by any other strategy.

Swap count. Number of steps in the decomposition, which bounds the number of passes over the buffer.

Auxiliary bytes. Peak cycle metadata, which the objectives require to stay independent of N .

Achieved throughput. Bytes moved divided by measured wall time, in GB/s. This is what calibrates μ and what exposes the gap between the cost model’s prediction and the hardware’s behaviour.

End-to-end latency and peak memory. Measured against the out-of-place baseline, which is a permute followed by a contiguous copy. That is the operation this work is meant to replace. Latency says whether the approach is competitive. Peak memory is the claim it is actually making.

3.5 Correctness

Every plan is checked against the framework’s own out-of-place permutation on the same input. A decomposition that moves the fewest bytes is worthless if it produces the wrong tensor, so correctness is a gate on every measurement rather than a separate experiment.

[PLACEHOLDER] State the shape and rank coverage of the correctness suite once it is written, as a generated quantity rather than a typed number.

4 Milestones and Timeline

Table 2 gives the plan, one milestone per week over ten weeks, with every milestone assigned to one of the two members named on the title page. The split follows the division of responsibilities stated there: the planner and cost-model milestones are Kaveen’s, the measurement and validation milestones are Alexis’s, and the milestones marked to both are genuinely joint work. Those are the comparison against prior systems and the write-up.

The ordering is not arbitrary: the calibration milestones come first because every later measurement is interpreted through the cost model, and a number produced before τ and μ are re-derived would have to be produced again afterwards.

Weeks are numbered from the week this proposal is submitted rather than dated, so the schedule can be anchored to the course calendar without rewriting it.

[PLACEHOLDER] Anchor the first week to a calendar date once the submission date is fixed.

References

- [1] Bryan Catanzaro, Alexander Keller, and Michael Garland. A decomposition for in-place matrix transposition. In *Proceedings of the 19th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP ’14)*, pages 193–206, Orlando, FL, USA, 2014. ACM. DOI <https://doi.org/10.1145/2555243.2555253>. Also issued as ACM SIGPLAN Notices 49(8):193–206.
- [2] Kai-Jung Cheng and Che-Rung Lee. ITTPD: In-place tensor transposition with permutation decomposition on GPUs. In *Proceedings of the International Conference on High Performance Computing in Asia-Pacific Region (HPCAsia ’25)*, pages 90–98, Hsinchu, Taiwan, 2025. DOI <https://doi.org/10.1145/3712031.3712033>.
- [3] David S. Dummit and Richard M. Foote. *Abstract Algebra*. John Wiley & Sons, Hoboken, NJ, 3rd edition, 2003. Section 1.3 gives the decomposition of a permutation into disjoint cycles, unique up to the order of the cycles.

Table 2: **Milestones, one per week.** 10 weeks from proposal to final report. Calibration is front-loaded, so that every measurement taken later is read through a cost model that has already been validated against the evaluation hardware.

Week	Milestone	Owner
1	Proposal submitted. Planner sweep committed and reproducible.	KJ, AN
2	Re-derive τ from a D_{post} micro-benchmark on the evaluation GPU, then restore graph pruning.	AN
3	Re-fit μ against the τ -pruned graph. Report the cost model’s error against measured throughput.	KJ
4	Close the fixed-point gap. Make search edge weights reflect real cycle structure and quantify how far the chosen plans move.	KJ
5	Correctness suite across ranks and shapes, gating every measurement that follows.	AN
6	End-to-end benchmark against the out-of-place baseline, measuring latency and peak memory.	AN
7	Head-to-head against the closest prior systems on their reported shapes.	KJ, AN
8	Derive D_{post}^* from the comparison and enable routing.	KJ
9	Draft the report, with every number generated from committed data.	KJ, AN
10	Final report and presentation.	KJ, AN

Weeks are numbered from the week the proposal is submitted. Owner initials are the team members named on the title page. AN is Alexis Newman and KJ is Kaveen Jayamanna.

- [4] Juan Gómez-Luna, I-Jui Sung, Li-Wen Chang, José María González-Linares, Nicolás Guil, and Wen-mei W. Hwu. In-place matrix transposition on GPUs. *IEEE Transactions on Parallel and Distributed Systems*, 27(3):776–788, 2016. DOI <https://doi.org/10.1109/TPDS.2015.2412549>.
- [5] Fred Gustavson, Lars Karlsson, and Bo Kågström. Parallel and cache-efficient in-place matrix storage format conversion. *ACM Transactions on Mathematical Software*, 38(3):17:1–17:32, 2012. Article 17. DOI <https://doi.org/10.1145/2168773.2168775>.
- [6] Adam Paszke. Add more Tensor methods, June 2016. PyTorch GitHub commit 60f9834, <https://github.com/pytorch/pytorch/commit/60f9834ac6ccb8721548e779d1d3db922d06e6e6>, authored 9 June 2016. Registers contiguous in the tensor method table of `torch/csrc/generic/Tensor.cpp`.
- [7] PyTorch Contributors. `torch.permute`. <https://docs.pytorch.org/docs/2.13/generated/torch.permute.html>, 2026. PyTorch 2.13 documentation. Accessed August 2026. Archived at <https://web.archive.org/web/20260808203715/https://docs.pytorch.org/docs/2.13/generated/torch.permute.html>.
- [8] PyTorch Contributors. `torch.transpose`. <https://docs.pytorch.org/docs/2.13/generated/torch.transpose.html>, 2026. PyTorch 2.13 documentation. Accessed August 2026. Archived at <https://web.archive.org/web/20260830062342/https://docs.pytorch.org/docs/2.13/generated/torch.transpose.html>.
- [9] PyTorch Team. PyTorch 2.x: Faster, more pythonic and as dynamic as ever. <https://pytorch.org/get-started/rtd/pytorch-2-x/>, 2026. Accessed August 2026. Archived at <https://web.archive.org/web/20260819091658/https://pytorch.org/get-started/pytorch-2-x/>.
- [10] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC20)*, pages 20:1–20:16. IEEE/ACM, 2020. Article 20. DOI <https://doi.org/10.1109/SC41405.2020.00024>. Also arXiv:1910.02054. The fragmentation claim is supported by its Section 3, “Managing fragmented Memory”.
- [11] I-Jui Sung, Geng Daniel Liu, and Wen-mei W. Hwu. DL: A data layout transformation system for heterogeneous computing. In *2012 Innovative Parallel Computing (InPar)*, pages 1–11. IEEE, May 2012. DOI <https://doi.org/10.1109/InPar.2012.6339606>.
- [12] Ke Wen. [RFC] communication primitives for non-contiguous tensors. <https://github.com/pytorch/pytorch/issues/177427>, March 2026. PyTorch GitHub issue #177427, opened 13 March 2026. Source of every RFC quotation in this paper. Accessed August 2026.
- [13] Chun-Yu Wu, Chih-Chieh Tu, Kai-Jung Cheng, and Che-Rung Lee. EITHOT: Efficient in-place transposition of high order tensors on GPUs. *ACM Transactions on Parallel Computing*, 12(1):1–22, February 2025. DOI <https://doi.org/10.1145/3711871>.